Load balancing and loop scheduling

Performance degradation

SAB: section 1.4.5

- A number of factors can contribute to real-world parallel performance being less than the idealised peak performance:

  - **S**tarvation: insufficient parallel work to keep the processors busy (insufficient parallel work, **uneven distribution of work**)

  - **L**atency: time taken for information to travel from one part of the system to another (e.g. accessing memory, sending messages)

  - **O**verhead: work required in addition to the computation (e.g. starting and stopping OpenMP parallel regions)

  - **W**aiting: multiple threads/processes accessing a shared resource (e.g. contention for memory or network bandwidth)

Lissajous program from workshop 2:

```
  /* Calculate values for x and y by looping over the parameter t */
#pragma omp parallel for default(shared) private(t)
  for (int i=0; i<N; i++){
    /* Convert i and N to floats so we do floating point division */
    t = 2.0*PI * (float) i / (float) N;
    x[i] = A * sin( omega_x*t + delta_x );
    y[i] = B * sin( omega_y*t + delta_y );
  }

  /* Write the results to a text file called output.dat */
  /* x and y are converted to pointers because they are arrays */
  write_to_file("output.dat", x, y, N);
```

How do we know that all the threads have finished writing to the arrays `x` and `y` before we call `write_to_file`?

Barriers and synchronisation

- To ensure correct functioning of a parallel code we may require that all threads reach a given point before proceeding
- In the Lissajous example all the threads must finish their calculations before the results are written out
- There is an implied barrier after parallel regions and work share constructs (e.g. `parallel for`) which acts as a synchronisation point

Explicit barriers and the `nowait` clause

- Sometimes we know that it is safe to remove an implied barrier
- The **`nowait`** clause removes an implied barrier

```
#pragma omp for nowait
```

- This can improve performance by avoiding unnecessary waiting but we need to be careful the program still works correctly
- Where synchronisation is required we add an explicit barrier

```
#pragma omp barrier
```

- All threads must reach the barrier otherwise deadlock will occur

Loop scheduling and load balancing

- If loop iterations do the same amount of work then all threads will reach the implied barrier around the same time
- If the amount of work per iteration varies some threads will finish before others and these threads will wait at the implied barrier
- We can control scheduling of loops to improve the load balancing and make better use of the resources

```
#pragma omp for schedule(...)
```

Scheduling options

- **`(static,`*`chunk`*`)`** - iterations divided into pieces of size chunk (or equal pieces if chunk is undefined) and distributed round-robin between threads
- (**`dynamic,`*`chunk`*`)`** - iterations divided into pieces of size chunk, when a thread finishes its chunk it is given another to work on
- (**`guided,`*`chunk`*`)`** - dynamic scheduling with decreasing chunk size. For **`chunk=1`** chunk size is proportional to the number of unassigned iterations divided by the number of threads in the team. **`chunk=k`** sets minimum chunk size

MPI and load balancing

- Load balancing is also a consideration for MPI parallelisation
- Blocking communication acts to synchronise MPI processes
- Blocking collectives synchronise all processes in the specified communicator
- With MPI we need to distribute the work between the available processes. This can be done in a way which load balances (e.g. manager-worker)

Unit summary

- There is an implied barrier at the end of parallel regions and work share constructs (e.g. **`parallel for`**)
- If the amount of work per loop iteration varies this can lead to some threads finishing before others and waiting at the barrier
- We can change the loop schedule to balance the load at run time using a **`dynamic`** or **`guided`** clause
- We will look at load balancing and loop scheduling with OpenMP in this week's workshop